VU AI HUB · LECTURE NOTES

Introduction to Python Programming for AI

Year 1, November to December 2025

Written by Inaaya and shared on Notion. Original page: cheddar-wall-f3b.notion.site/2999801a9309804ca1fafd8390797b88

Contents

1. 27th of October
2. 29th of October
3. 3rd of November
4. 5th of November
5. 10th of November
6. 12th of November
7. 17th of November
8. 19th of November
9. 24th of November
10. 26th of November
11. 1st of December
12. 3rd of December
13. 4th of December

27th of October

How you write the program - *syntax*

What do the symbols mean - *semantics*

How people use the programming language - *pragmatics*

- ex. spaces between symbols don't matter when you run the program

Pythonic - what matches the best with python and is better and nicer

---

Tools to develop software

- Editors - notebooks, VScode
- Integrated Development Environment (IDE) - settings of your own computer?
- Compilers / interpreters - the commands we type are translated into machine language through these
- Libraries - base of programming
- Hardware - laptop, calculator etc.

> *A program is a sequence of instructions written in a programming language that specifies how to solve a problem*

Input → Instructions (Repetition, Match, Conditional execution) → Output

Computers only understand machine language → Binary code

String - a collection of any type of characters between double quotes " "

Functions - a bunch of characters followed by parantheses ( )

Built-in functions:

- PRINT - running whatever is between the ( )

- INPUT - after running the user will be able to answer and the rest of the code will be stopped because its waiting for the input of the user, there will never be an output if there is no input given

---

The difference between writing a program and executing a program is that a written program has not been run, by running the program you can check if it works

29th of October

Variables, Expressions, Assignments (and Functions)

Main building blocks for creating programs:

- Variables - store values that can be changed
- Expressions - constructs of values and variables with operators
- Statements - instructions with actual effects
- Functions - reusable blocks of code that perform specific tasks

---

Values and Types

A value is the basic unit used in a program

Each value has its own type:

- 42 is an integer (int)
- 42.0 is a floating-point number (float)
- 'Hello, Data Scientists!' is a string (str)

The `type` function will tell you what type your code is

- print (type(42)) → <class 'int'>

Expressions

An expression is a combination of values, variables, and operators

A value by itself is an expression, and so is a variable

Expressions are evaluated by the interpreter

*In Jupyter notebooks the last line of code will be printed, even if the print function is not written

---

Statements

A statement is an instruction that has an effect, like creating a variable or displaying a value

---

Assignment Statements

An assignment statement creates a new variable with a given name and assigns it a value

---

Type Hints

You can add type hints to make your code more readable and help with documentation

#after the hashtag you can put anything to explain your code and make everything clear without it interrupting your code

---

Variable Naming Rules

- Start with a letter or underscore
- Cannot start with a number
- Only contain alphanumeric characters and underscores
- Cannot be Python keywords that already have a specific function in your code cannot be a variable name
- Be meaningful and self-documenting

Python Keywords

`False` `class` `finally` `is` `return` `None` `continue` `for` `lambda` `try`
`True` `def` `from` `nonlocal` `while` `and` `del` `global` `not` `with`
`as` `elif` `if` `or` `yield` `assert` `else` `import` `pass` `break`
`except` `in` `raise`

Reassignments

You can assign a new value to an existing variable. This process is called reassignment. The old value is lost when you assign a new value

---

Updating Variables

A frequently used reassignment is for updating purposes: the value of a variable depends on the previous value

---

Operations

Shorthand Operators

Use `+=` and `-=` for increment and decrement operations

- z += 100 #same as z = z + 100, and the same for -=

Arithmetical Operators

- Addition = +
- Subtraction = -
- Multiplication = *
- Division = /
- Exponentiation = **
- Floor division = // #divides two numbers and rounds down to an integer (555//60 = 9)
- Modulus Operator = % #computes the remainder when dividing the first integer by the second (555%60 = 15)

Order of Operations

PEMDAS

1. **P**arentheses
2. **E**xponentiation
3. **M**ultiplication and **D**ivision
4. **A**ddition and **S**ubtraction

String Operations

You cannot perform most mathematical operations on strings, but there are two exceptions

- `+` performs string concatenation
  - first: str = 'data'
  - second: str = ' '
  - third: str = 'science'
  - first + second + third # 'data science'
- `*` performs repetition
  - ('Data' + ' ') * 3 # 'Data Data Data '

User Input

Use the `input()` function to get data from the user, the program will stop and wait for the user input

Type Conversion

Convert string input to other types using `int()` and `float()` functions

- age = input ('What is your age?') #user puts in 18
- print (int(age)) #it will convert the 18 to an integer, 18
- print (float(age)) #it will convert the 18 to a float, 18.0

Introduction to Functions

A function is a reusable block of code that performs a specific task

Think of functions as mini-programs within your program that you can call whenever needed

- def greet ( ) : #define a simple function
  - print("Hello from a function!")
- greet ( ) #call the function

Pros functions

- Avoid repetition: Write code once, use it many times, only have to change it at one spot
- Organize code: Break complex problems into smaller, manageable pieces
- Make code readable: Give meaningful names to code blocks
- Debug easily: Test and fix individual pieces of functionality
- Reuse code: Share functions across different programs

Function Arguments

Arguments are inputs that you pass to a function, they're between parentheses ( )

They allow functions to work with different data each time they're called

---

Return Values

Functions can return a value back to the code that called them

The returned value can be stored in a variable or used directly

It has nothing to do with what you see when you print something, it'll just tell you what your function does

---

Function Call vs Function

The function name is variable with the function as a value

- print(len) #This is a reference to the function

Use parentheses to call the function and execute the code

- print(len("Add one string")) #this calls the `len` function and will print the number of characters in that sentence, 14

Sources of Functions

Built-in Functions

Python comes with many useful functions already built-in

- `print()` - displays output
- `input()` - gets user input
- `len()` - returns length of a string or list
- `type()` - returns the type of a value
- `int()`, `float()`, `str()` - convert between types

Imported Functions

You can import functions from external libraries using the `import` statement

This gives you access to thousands of additional functions for specific tasks

- help(math), this for example gives you all kinds of codes you can use that have to do with math

User-Defined Functions

You can create your own functions using the `def` keyword

This allows you to write custom code that you can reuse throughout your program

- def add (x, y)
- return x + y
- message = add (2, 6)
- print (message)
  - 8

Script Mode vs Interactive Mode

Interactive Mode (Jupyter Notebooks)

- Good for learning and experimentation
- Run code cell by cell

Script Mode (Python files)

- Save code in `.py` files
- Run entire programs at once
- Better for larger projects

3rd of November

Control Flow Statements (Conditional)

Python programs execute from top to bottom

Boolean Values

It's an expression that's either false or true

You can't use `True` or `False` for naming anything, because they already have a function in Python

Comparison Operators

Operator	Meaning
==	equal to
!=	not equal to
<	less than
>	greater than
<=	less than or equal to
>=	greater than or equal to

Every time you use a comparison operator you are obtaining a boolean value, the result of this operation will either be `True` or `False`

- `=` is not a comparison operator, it's an assignment operator and used for assigning the value on the left to a variable on the right.

If you ask for input from the user you will always get it back as a string, even if they respond with numbers, it will appear in quotation marks `" "` and not be seen as an integer. A way to bypass this is to name the input `a`, for example, and the next line is something like `a_int = int(a)`, it will change the string to an integer, and this way you can use the comparison operators (because `42 == "42"` will output `False`).

`42 == 42.0` however will output `True`

Boolean Operators

- `and` is a binary operator, it will only output `True` if both sides are true, if just one side or no sides are true it will output `False`.
  - Both sides always need to be a boolean value
- `or` is a binary operator, it will output `True` if both sides and if just one side is true, only when both sides are false it will output `False`.
- `not` is a unary operator (there is only one side you need to check), when that is not true it will output `False` and when it is not false it will output `True`

Control Flow Components

Conditions

Always evaluated to a boolean value

- "If this condition is true, do this thing, else do this other thing"

Clauses (blocks of code)

It's a set of lines of code, a new block begins with indentation and ends when the indentation decreases. Blocks can contain other blocks. Python expects a new block after any statement that ends with a colon `:`

```
def name () :
    new block
```

An `if` statements' clause will be executed if the statements' condition is `True`. Otherwise, the clause is skipped.

An `else` statement can only be there if there is an `if` statement before

An `elif` statement is an optional statement you can use to follow an `if` statement, you use it when you want to execute one of many possible clauses

- A computer will always check all the `if` statements, so if you use another `if` instead of `elif` it will check everything, if you just use the `elif` it will stop checking after finding the true statement.

*All `if`, `else` and `elif` must be followed with a colon `:`

5th of November

Control Flow: Loops

While Loop Statement

You make a block of code execute over and over using while statement

A `while` statement consists of:

- `while` keyword
- A condition
- A colon (:)
- A block of code or `while` clause

→ `while [condition] :`

Break statement

Break out a while loop's clause early

Continue statement

"Hey..... I reached a condition I don't want to break or go out the loop but want to go back and check the condition"

~ Manbun guy, 2025

For Loop Statement

You can write the same code as the while loop, but you can put a limit to it

A `for` statement consists of:

- The `for` keyword
- A variable name
- The `in` keyword
- A call to the `range()` function
- A colon (:)
- A block or `for` clause

→ `for [Var in range()]:`

Range(U) - 0 to upper limit

Range(L, U) - lower to upper limit

Range(L, U, S) - lower to upper limit, but not by 1, you can define how you want to count up

*The U, L and S are substitutes for numbers you put in yourself

10th of November

Functions

There are different kind of functions:

- Built-in functions
- Imported functions
- User-defined functions
  - The major purpose of functions is to group code that gets executed multiple times

How to define a function:

- def statement (function signature)
- colon (:)
- function body

*After the colon there is always an indentation

How to call a function:

- `print(name_function)`
  - will print the answer and `None`
- `name_function()`
  - will just print the answer

Arguments and Parameters

- `print()`
- `len()`

Inside the function, the arguments are assigned to variables called parameters

```
def print_twice(bruce):
    print(bruce)
    print(bruce)
```

'bruce' is the parameter

Return values and return Statements

Important Terms

- define
- call,
- pass,
- argument, and
- parameter

When creating a function, you can specify the return value, which consists of:

- A return keyword
- The value or expression that the function should return

Void functions - `→ None:`

- It represents the absence of a value

Fruitful functions - ex. `→ int:` , `→ str:` , `→ float:` , `→ bool:`

*When you return the value in defining the function instead of print, you can reuse the value you generated and you print the return function and do something else as well, ex:

```
def sum_int(num_a: int, num_b: int) → int:
    return(num_a + num_b)
sum_int(9, 7) * 6
```

96

12th of November

Functions

Named Parameters

Python identifies most arguments by their position in the function call

Separators

`sep = “, “` = a comma in between arguments

`end = “\n”` = enter, new line

`end = “\t”` = tab

Scoping

All the indentations underneath a functions are within the scope of the function. These lines of code only exist within the scope of the function, outside of the function they don’t exist.

Lists

A list is a value that contains multiple values (items) in an ordered sequence

A list begins with an opening square bracket and ends with a closing square bracket, [ ]

By using these brackets you change multiple values to 1 value

`spam = [’cat’, False, ‘bat’, ‘rat’, ‘elephant’]`

`spam[0]` → will print cat

`type(spam[1])` → will print bool, because False is a boolean, that’s the type

17th of November

Dictionaries

`del` → delete something

- `del list[2]`
  - Delete the second element of the list

`in` and `not` Operators

- you can determine whether a value is or isn’t in a list with these operators

`.append` adds a new element to an existing list

Dictionaries allow you to store data as key-value pairs, making it easy to look up information by name instead of position

- In Python you create dictionaries by using curly brackets `{}`

Loop through keys `(.keys())`

```
for key in my_cat.keys():
    print(my_cat[])
```

Loop through values `(.values())`

Loop through both keys and values `(.items())`

19th of November

Nested dictionaries

Dictionaries can contain other dictionaries - this is useful for structured data

24th of November

oops

26th of November

Classes and Methods

Classes

A class is a programmer-defined type. A class definition creates a new class object

- Objects (instances)

A method almost always has a self in there, otherwise it's probably just a function

1st of December

Inheritance

3rd of December

Reading and Writing Files

Files are the primary way that programs interact with the outside world

File Paths

A path is a string representing the location of a file or folder

`".my_ipynb"` to open the file if it's at the current level

`".Python\my_ipynb"` to open a file in a folder

→ You write these as strings

→ Windows uses backslashes `\`

4th of December

Python Course for Beginners